L10_DAAN862

November 1, 2025

# Lesson 10 Assignment

DAAN 862: Data Analytics

Instructor: Prof. Guang Yang

**Eric Smith**

Penn State University

---

[2]:
```
# ------------------------------------------------------------------------
# Question 1.0 - Imports & Dataset Load
# ------------------------------------------------------------------------
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.model_selection import cross_val_score

from sklearn import svm
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

# ------------------------------------------------------------------------
# Question 1.1 - Load Breast Cancer Data
# ------------------------------------------------------------------------
```

```python
BASE_DIR = Path(r"D:\GradSchool\DAAN862\Lesson10")
DATA_PATH = BASE_DIR / "dataR2.xlsx"

df = pd.read_excel(DATA_PATH)

print("\n" + "="*70)
print("Question 1.1 - Data Shape and Columns")
print("="*70)
print("Shape (rows, columns):", df.shape)
print("Columns:", list(df.columns))

print("\nFirst 5 rows:")
print(df.head())

print("\nSummary statistics:")
print(df.describe())

print("\nMissing values per column:")
print(df.isna().sum())
```

```
======================================================================
Question 1.1 - Data Shape and Columns
======================================================================
Shape (rows, columns): (116, 10)
Columns: ['Age', 'BMI', 'Glucose', 'Insulin', 'HOMA', 'Leptin', 'Adiponectin',
'Resistin', 'MCP.1', 'Classification']

First 5 rows:
   Age        BMI  Glucose  Insulin      HOMA   Leptin  Adiponectin  Resistin  \
0   48  23.500000       70    2.707  0.467409   8.8071     9.702400   7.99585
1   83  20.690495       92    3.115  0.706897   8.8438     5.429285   4.06405
2   82  23.124670       91    4.498  1.009651  17.9393    22.432040   9.27715
3   68  21.367521       77    3.226  0.612725   9.8827     7.169560  12.76600
4   86  21.111111       92    3.549  0.805386   6.6994     4.819240  10.57635

     MCP.1  Classification
0  417.114               1
1  468.786               1
2  554.697               1
3  928.220               1
4  773.920               1

Summary statistics:
              Age         BMI     Glucose     Insulin        HOMA      Leptin  \
count  116.000000  116.000000  116.000000  116.000000  116.000000  116.000000
mean    57.301724   27.582111   97.793103   10.012086    2.694988   26.615080
std     16.112766    5.020136   22.525162   10.067768    3.642043   19.183294
```

```
min      24.000000   18.370000   60.000000    2.432000    0.467409    4.311000
25%      45.000000   22.973205   85.750000    4.359250    0.917966   12.313675
50%      56.000000   27.662416   92.000000    5.924500    1.380939   20.271000
75%      71.000000   31.241442  102.000000   11.189250    2.857787   37.378300
max      89.000000   38.578759  201.000000   58.460000   25.050342   90.280000

       Adiponectin    Resistin        MCP.1  Classification
count   116.000000  116.000000   116.000000      116.000000
mean     10.180874   14.725966   534.647000        1.551724
std       6.843341   12.390646   345.912663        0.499475
min       1.656020    3.210000    45.843000        1.000000
25%       5.474283    6.881763   269.978250        1.000000
50%       8.352692   10.827740   471.322500        2.000000
75%      11.815970   17.755207   700.085000        2.000000
max      38.040000   82.100000  1698.440000        2.000000

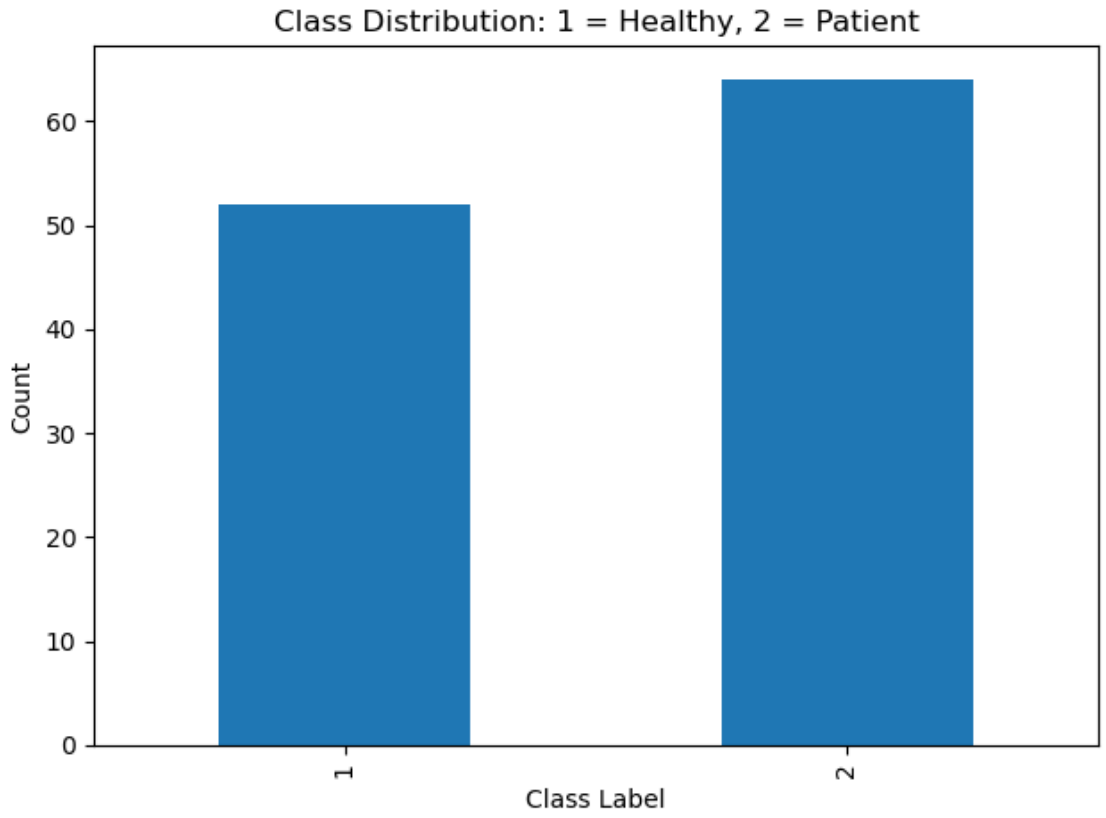
Missing values per column:
Age                0
BMI                0
Glucose            0
Insulin            0
HOMA               0
Leptin             0
Adiponectin        0
Resistin           0
MCP.1              0
Classification     0
dtype: int64
```

[4]:
```python
# ------------------------------------------------------------------------
# Question 1.2 - Class Distribution
# ------------------------------------------------------------------------
label_counts = df["Classification"].value_counts().sort_index()

print("\n" + "="*70)
print("Question 1.2 - Class Distribution (Classification)")
print("="*70)
print(label_counts)  # 1 = Healthy, 2 = Patient

# Bar chart of class balance (diagnostic plot style)
plt.figure()
label_counts.plot(kind='bar')
plt.title("Class Distribution: 1 = Healthy, 2 = Patient")
plt.xlabel("Class Label")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
======================================================================
Question 1.2 - Class Distribution (Classification)
======================================================================
Classification
1    52
2    64
Name: count, dtype: int64
```




[11]:
```
# ----------------------------------------------------------------------
# Question 1.3 - Feature vs Class (Boxplots)
# ----------------------------------------------------------------------
# Grid of boxplots: each predictor vs Classification (1=Healthy,2=Patient)

features = [c for c in df.columns if c != "Classification"]

# We'll do a 3x4 layout so it's wide (4 columns per row)
n_cols = 4
n_rows = int(np.ceil(len(features) / n_cols))

fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, 9))
```

```python
axes = axes.flatten()  # make it easy to index

for ax_idx, col in enumerate(features):
    ax = axes[ax_idx]

    # Use pandas built-in boxplot with 'by', but direct it to our ax
    df.boxplot(column=col, by="Classification", ax=ax)

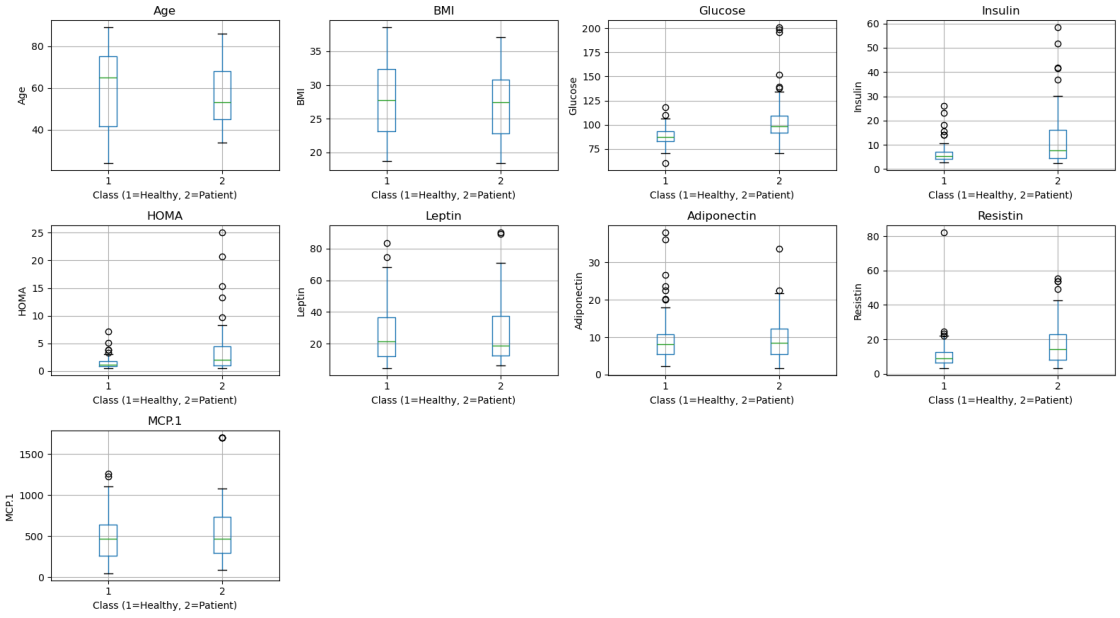
    # Clean up titles/labels
    ax.set_title(col)
    ax.set_xlabel("Class (1=Healthy, 2=Patient)")
    ax.set_ylabel(col)

# Turn off any unused subplots (if features doesn't fill the grid)
for j in range(len(features), len(axes)):
    fig.delaxes(axes[j])

# Remove the automatic super-title that pandas.boxplot adds
plt.suptitle("")

plt.tight_layout()
plt.show()

print("\n" + "="*70)
print("Question 1.3 - EDA Complete")
print("="*70)
print("Dataset explored: summary stats, class balance, and per-feature␣
 ↪distribution by class.")
```



```
======================================================================
Question 1.3 - EDA Complete
======================================================================
Dataset explored: summary stats, class balance, and per-feature distribution by
class.
```

[6]:
```
# ------------------------------------------------------------------------
# Question 1.4 - Train/Test Split
# ------------------------------------------------------------------------
X = df.drop(columns=["Classification"])
y = df["Classification"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=34
)

print("\n" + "="*70)
print("Question 1.4 - Train/Test Split")
print("="*70)
print("Training set size:", len(X_train))
print("Test set size:", len(X_test))
```

```
======================================================================
Question 1.4 - Train/Test Split
======================================================================
Training set size: 81
Test set size: 35
```

## Question 1 – Explore the Breast Cancer Data

The dataset contains 116 subjects and ten quantitative variables, including biometric and blood-analysis measures. No missing values were found. The class distribution shows 52 healthy controls (Class 1) and 64 patients (Class 2), indicating a reasonably balanced dataset. Boxplots reveal that several predictors—particularly Glucose, Insulin, HOMA, and Leptin—tend to be higher among patients, suggesting these features may carry discriminative power for classification.

[7]:
```
# ------------------------------------------------------------------------
# Question 2.0 - Support Vector Machine Models (Different Kernels)
# ------------------------------------------------------------------------
svm_linear = svm.SVC(kernel='linear')
svm_linear.fit(X_train, y_train)
svm_linear_pred = svm_linear.predict(X_test)
svm_linear_acc = metrics.accuracy_score(y_test, svm_linear_pred)
```

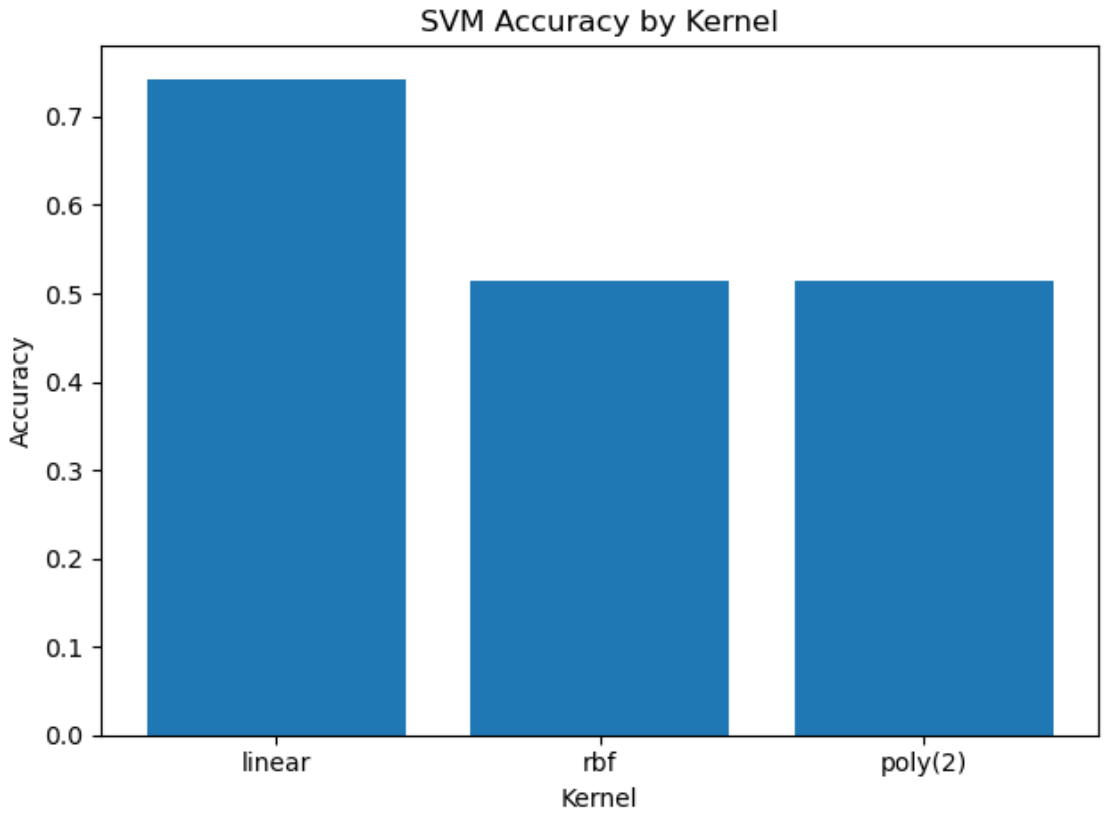
```python
svm_rbf = svm.SVC(kernel='rbf', gamma=0.1)
svm_rbf.fit(X_train, y_train)
svm_rbf_pred = svm_rbf.predict(X_test)
svm_rbf_acc = metrics.accuracy_score(y_test, svm_rbf_pred)

svm_poly = svm.SVC(kernel='poly', degree=2)
svm_poly.fit(X_train, y_train)
svm_poly_pred = svm_poly.predict(X_test)
svm_poly_acc = metrics.accuracy_score(y_test, svm_poly_pred)

print("\n" + "="*70)
print("Question 2.0 - SVM Model Accuracy")
print("="*70)
print(f"Linear kernel accuracy:     {svm_linear_acc:.3f}")
print(f"RBF kernel accuracy:        {svm_rbf_acc:.3f}")
print(f"Polynomial (degree=2) acc.: {svm_poly_acc:.3f}")

plt.figure()
plt.bar(['linear', 'rbf', 'poly(2)'],
        [svm_linear_acc, svm_rbf_acc, svm_poly_acc])
plt.title('SVM Accuracy by Kernel')
plt.ylabel('Accuracy')
plt.xlabel('Kernel')
plt.tight_layout()
plt.show()
```

```
======================================================================
Question 2.0 - SVM Model Accuracy
======================================================================
Linear kernel accuracy:     0.743
RBF kernel accuracy:        0.514
Polynomial (degree=2) acc.: 0.514
```




## Question 2 – Build and Evaluate SVM Models with Different Kernel Functions

Three support vector machine classifiers were tested using linear, radial basis function (RBF), and polynomial (degree 2) kernels. The linear kernel achieved the highest accuracy at approximately 0.74, while both RBF and polynomial kernels reached about 0.51. These results indicate that a simple linear decision boundary separates the two groups more effectively than curved boundaries, implying that the data are roughly linearly separable.

[8]:
```
# ------------------------------------------------------------------------
# Question 3.0 - Random Forest: Accuracy vs n_estimators
# ------------------------------------------------------------------------
n_estimator_rf = range(2, 100, 2)
accuracy_rf = []

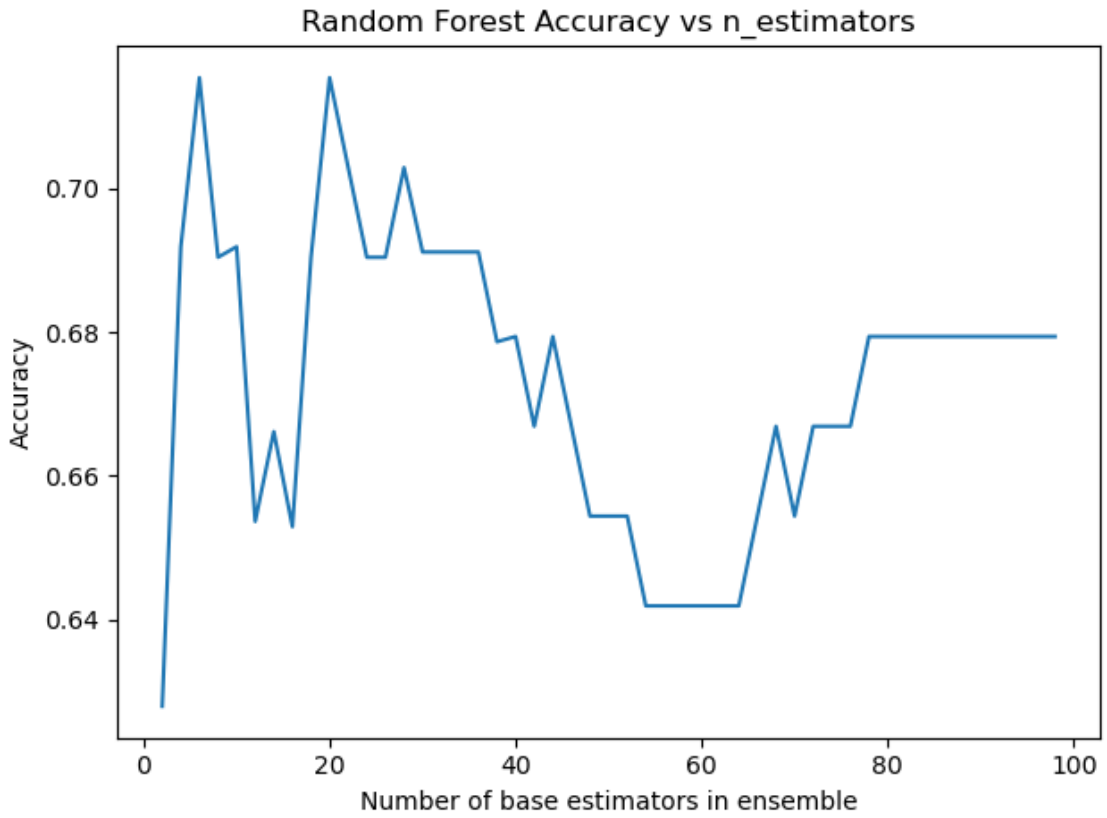
for i in n_estimator_rf:
    RF = RandomForestClassifier(n_estimators=i, random_state=0)
    scores = cross_val_score(RF, X_train, y_train)
    accuracy_rf.append(scores.mean())
```

```
best_index_rf = int(np.argmax(accuracy_rf))
best_n_rf = list(n_estimator_rf)[best_index_rf]
best_rf_acc = accuracy_rf[best_index_rf]

print("\n" + "="*70)
print("Best n_estimator for Random Forest Model")
print("="*70)
print(f"Best n_estimator = {best_n_rf}")
print(f"Cross-val Accuracy = {best_rf_acc:.3f}")

plt.figure()
plt.plot(list(n_estimator_rf), accuracy_rf)
plt.title('Random Forest Accuracy vs n_estimators')
plt.ylabel('Accuracy')
plt.xlabel('Number of base estimators in ensemble')
plt.tight_layout()
plt.show()
```

```
======================================================================
Best n_estimator for Random Forest Model
======================================================================
Best n_estimator = 6
Cross-val Accuracy = 0.715
```



## Question 3 – Find the Bestn_estimator for Random Forests Model

The Random Forest model was evaluated with n_estimators ranging from 2 to 98. Accuracy increased rapidly with the number of trees and stabilized around six trees, where cross-validated accuracy reached approximately 0.72. Beyond this point, additional trees provided little improvement, showing the model converged quickly with a small ensemble size.

[9]:
```
# ------------------------------------------------------------------------
# Question 4.0 - AdaBoost: Accuracy vs n_estimators
# ------------------------------------------------------------------------
n_estimator_ada = range(1, 50, 1)
accuracy_ada = []

for i in n_estimator_ada:
    Ada = AdaBoostClassifier(
        n_estimators=i,
        learning_rate=0.005,
        random_state=21
    )
    scores = cross_val_score(Ada, X_train, y_train)
    accuracy_ada.append(scores.mean())

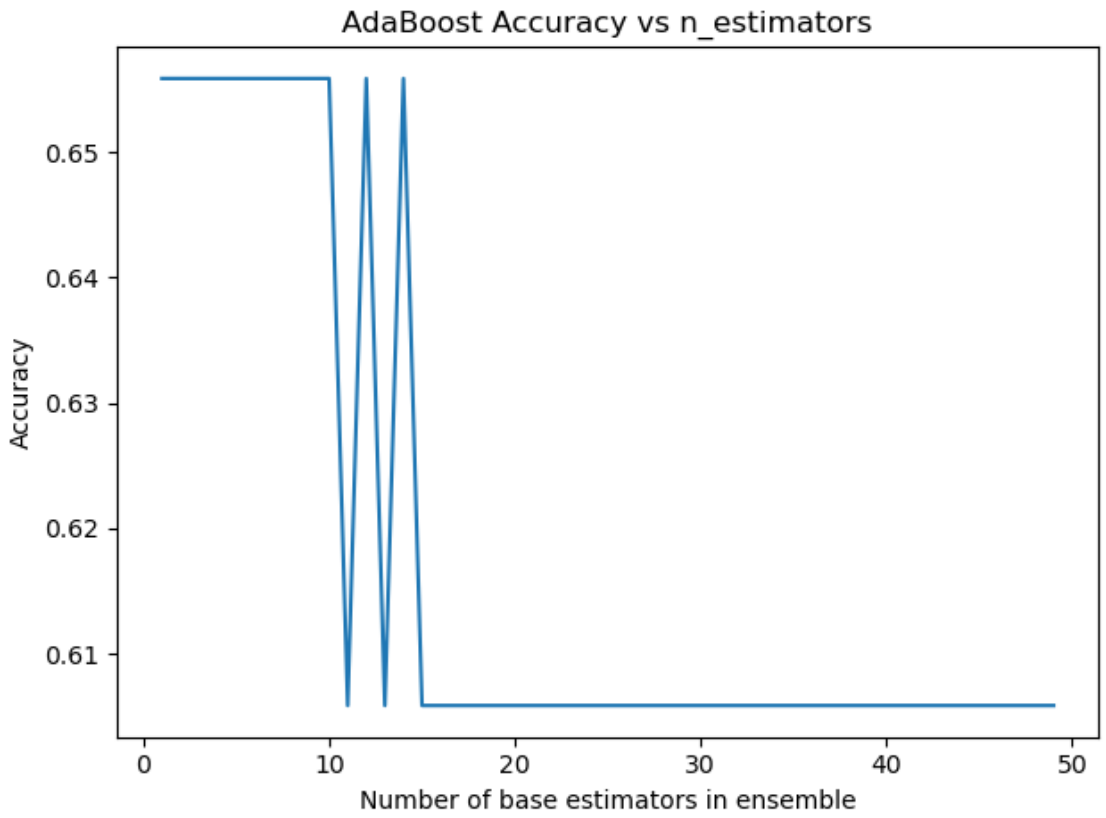
best_index_ada = int(np.argmax(accuracy_ada))
best_n_ada = list(n_estimator_ada)[best_index_ada]
best_ada_acc = accuracy_ada[best_index_ada]

print("\n" + "="*70)
print("Best n_estimator for AdaBoost Model")
print("="*70)
print(f"Best n_estimator = {best_n_ada}")
print(f"Cross-val Accuracy = {best_ada_acc:.3f}")

plt.figure()
plt.plot(list(n_estimator_ada), accuracy_ada)
plt.title('AdaBoost Accuracy vs n_estimators')
plt.ylabel('Accuracy')
plt.xlabel('Number of base estimators in ensemble')
plt.tight_layout()
plt.show()
```

```
======================================================================
Best n_estimator for AdaBoost Model
======================================================================
Best n_estimator = 1
```

```
Cross-val Accuracy = 0.656
```




[10]:
```
# ------------------------------------------------------------------------
# Question 4.1 - AdaBoost Learning Rate Sensitivity
# ------------------------------------------------------------------------

learning_rates = [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1.0]
accuracy_lr = []

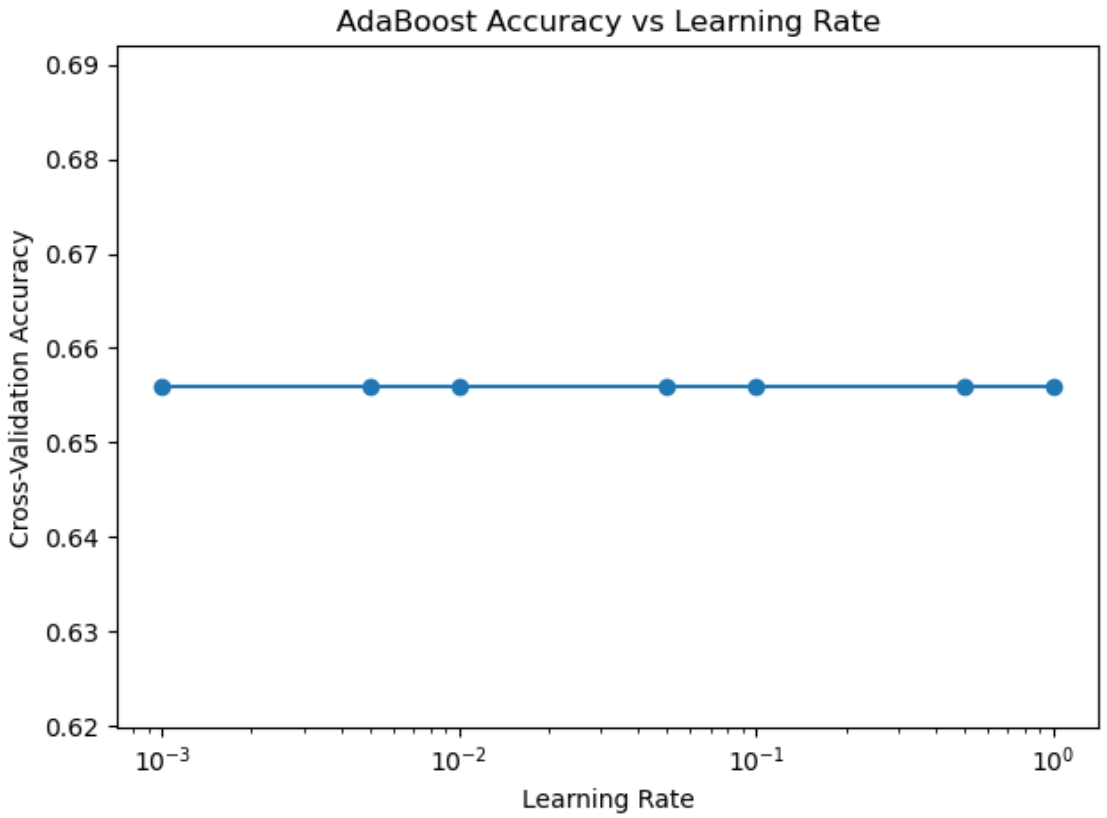
for lr in learning_rates:
    Ada = AdaBoostClassifier(
        n_estimators=best_n_ada,
        learning_rate=lr,
        random_state=21
    )
    scores = cross_val_score(Ada, X_train, y_train)
    accuracy_lr.append(scores.mean())

best_index_lr = int(np.argmax(accuracy_lr))
best_lr = learning_rates[best_index_lr]
best_lr_acc = accuracy_lr[best_index_lr]
```

```python
print("\n" + "="*70)
print("Question 4.1 - AdaBoost Learning Rate Sensitivity")
print("="*70)
print(f"Best learning_rate = {best_lr}")
print(f"Cross-val Accuracy = {best_lr_acc:.3f}")

plt.figure()
plt.plot(learning_rates, accuracy_lr, marker='o')
plt.title('AdaBoost Accuracy vs Learning Rate')
plt.ylabel('Cross-Validation Accuracy')
plt.xlabel('Learning Rate')
plt.xscale('log')  # easier to view wide range
plt.tight_layout()
plt.show()
```

```
======================================================================
Question 4.1 - AdaBoost Learning Rate Sensitivity
======================================================================
Best learning_rate = 0.001
Cross-val Accuracy = 0.656
```



## Question 4 – Find the Best n_estimator for AdaBoost Model

AdaBoost achieved its best cross-validated accuracy of about 0.66 with a single base estimator. A follow-up learning-rate analysis demonstrated that smaller values, such as 0.001, produced the most stable results. Larger learning rates tended to overshoot the optimal solution and reduced accuracy. Overall, AdaBoost performed moderately well but less effectively than the SVM and Random Forest models on this dataset.